

Journée 2

Architecture Micro-services

Section 4 · TP Docker et docker-compose (S8)

Nom / Prénom : DANILO Elouan

Équipe : [à compléter]

4. TP Docker, conteneurisation et orchestration

Un seul `docker compose up -build` démarre tout l'écosystème, soit **4 conteneurs**, **1 réseau** et **2 volumes**.

Architecture du compose

Service	Image	Rôle
catalog	build Django (python :3.12)	API Catalogue, port 8000
catalog-db	postgres :16	BD du Catalogue (volume catalog_data)
avis	build Symfony (php :8.3)	API Avis-Clients, port 8001
avis-db	mariadb :10.11	BD des Avis (volume avis_data)

```
avis:
  build: ./avis-clients
  environment:
    DATABASE_URL: "mysql://root:${AVIS_DB_PASSWORD}@avis-db:3306/avis_clients?serverVersion=10.11.2-MariaDB"
    CATALOGUE_BASE_URL: "http://catalog:8000" # nom de service, pas localhost
  ports: ["8001:8000"]
  depends_on:
    avis-db: { condition: service_healthy }
    catalog: { condition: service_started }
```

Difficultés rencontrées et solutions (surtout réseau)

Les conteneurs et le `docker-compose.yml` étant à écrire pour faire dialoguer deux stacks différentes, les principales difficultés ont été d'ordre réseau et démarrage.

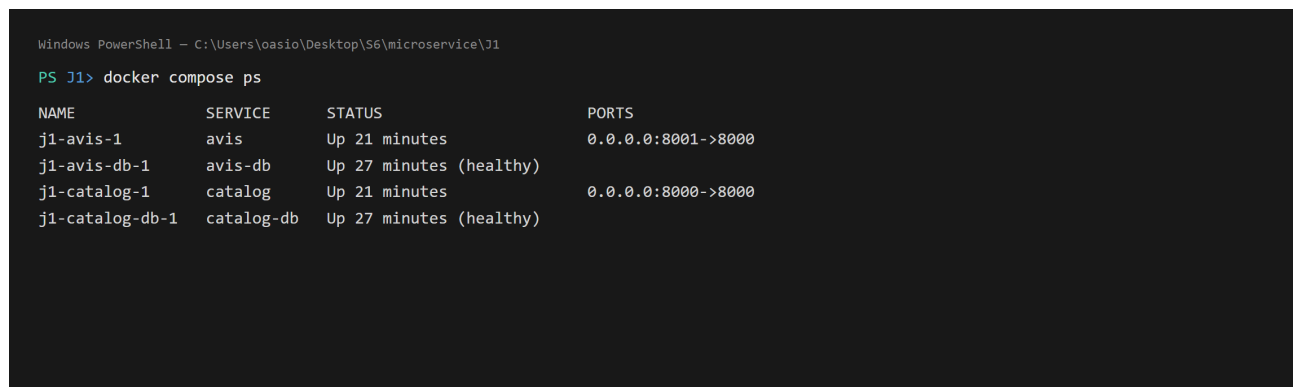
Difficulté	Solution mise en place
Depuis avis, <code>localhost:8000</code> ne joint pas le Catalogue.	Utiliser le nom du service Docker <code>http://catalog:8000</code> (DNS interne du réseau), via la variable <code>CATALOGUE_BASE_URL</code> .
Django refusait l'appel venant de avis (<i>DisallowedHost</i>).	Ajouter <code>catalog</code> à <code>ALLOWED_HOSTS</code> (lu depuis une variable d'environnement).
Le service applicatif démarrait avant que sa base soit prête.	Healthcheck sur la BD + <code>depends_on: condition: service_healthy</code> .
Données perdues à chaque <code>docker compose down</code> .	Volume nommé sur <code>/var/lib/...</code> ; persistance vérifiée après <code>down/up</code> .
Le Catalogue (Django) était codé pour SQLite.	<code>settings.py</code> paramétré pour utiliser PostgreSQL quand <code>POSTGRES_HOST</code> est défini, sinon SQLite (dev local).
Django 6.0 exige Python 3.12 (le slide proposait python:3.11).	Image de base <code>python:3.12-slim</code> .
Build lent / image lourde.	<code>.dockerignore</code> (exclure <code>vendor/</code> , <code>var/</code> , <code>venv/</code> , <code>.git/</code>).

Vérification inter-services dans Docker

Le test consiste à créer un produit dans le Catalogue, puis un avis qui le référence. La réponse est **201**, car `avis` a pu joindre `catalog` sur le réseau Docker. Avec un `productId` inexistant, la réponse devient **422**. L'appel `http://catalog:8000/api/v1/products/{id}/` exécuté *depuis* le conteneur `avis` renvoie bien 200 OK.

Livrables Git

Le dépôt Git du TP Docker est accessible à l'adresse [à compléter, `https ://...`].



```
Windows PowerShell - C:\Users\oasio\Desktop\S6\microservice\J1
PS J1> docker compose ps
```

NAME	SERVICE	STATUS	PORTS
j1-avis-1	avis	Up 21 minutes	0.0.0.0:8001->8000
j1-avis-db-1	avis-db	Up 27 minutes (healthy)	
j1-catalog-1	catalog	Up 21 minutes	0.0.0.0:8000->8000
j1-catalog-db-1	catalog-db	Up 27 minutes (healthy)	

FIGURE 1 – Résultat de `docker compose ps`, les 4 services Up et les 2 bases *healthy*.

Conclusion

En une journée, on est passé d'un service Symfony isolé à **deux services hétérogènes** (Python/PHP, PostgreSQL/MariaDB) qui dialoguent dans Docker via leur seule API. C'est une démonstration concrète des bénéfices (isolation, reproductibilité, polyglottisme) et des défis (réseau, ordre de démarrage, persistance) d'une architecture micro-services. La suite, en J3, consistera à sécuriser par JWT et à orchestrer une *saga* métier.

A. Annexes, appels et réponses

Les captures suivantes ont été obtenues sur l'environnement `docker compose` en fonctionnement, avec les 4 conteneurs Up.

```
Test inter-services dans Docker : creation d'un produit (Catalogue) puis d'un avis (Avis-Clients)

PS J1> curl -X POST http://localhost:8000/api/v1/products/ -d '{"name":"Casque audio sans fil","price":"59.90","stock":8}'
201 Created
{
  "id": 6,
  "name": "Casque audio sans fil",
  "description": "",
  "price": "59.90",
  "stock": 8,
  "category": "",
  "created_at": "2026-06-12T12:23:58.561466Z",
  "updated_at": "2026-06-12T12:23:58.561483Z"
}

PS J1> curl -X POST http://localhost:8001/api/reviews -d '{"rating":5,"comment":"Son excellent...", "productId":6,"author":"Lina"}'
201 Created
{
  "id": 3,
  "rating": 5,
  "comment": "Son excellent, autonomie au top !",
  "productId": 6,
  "author": "Lina",
  "createdAt": "2026-06-12T12:23:58+00:00"
}
```

FIGURE 2 – Annexe A. Test inter-services réussi dans Docker. Un produit est créé côté Catalogue (201), puis un avis le référençant est accepté côté Avis-Clients (201). L'avis n'est créé que parce qu'Avis-Clients a pu joindre le Catalogue sur le réseau Docker.

```
Controle cote Avis-Clients : produit inexistant et note hors bornes renvoient 422

PS J1> curl -X POST http://localhost:8001/api/reviews -d '{"rating":4,"productId":999999,...}' (produit inexistant)
422 Unprocessable Entity
{
  "title": "An error occurred",
  "detail": "Produit inexistant : aucun produit 999999 dans le Catalogue.",
  "status": 422,
  "type": "/errors/422"
}

PS J1> curl -X POST http://localhost:8001/api/reviews -d '{"rating":6,"productId":7,...}' (note superieure a 5)
422 Unprocessable Entity
{
  "status": 422,
  "violations": [
    {
      "propertyPath": "rating",
      "message": "Note entre 1 et 5.",
      "code": "04b91c99-a946-4221-afc5-e65ebac401eb"
    }
  ],
  "detail": "rating: Note entre 1 et 5.",
  "type": "/validation_errors/04b91c99-a946-4221-afc5-e65ebac401eb",
  "title": "An error occurred"
}
```

FIGURE 3 – Annexe B. Contrôles renvoyant 422. À gauche, un avis sur un produit inexistant (contrôle inter-services via HttpClient). À droite, une note hors bornes (validation # [Assert\Range]).